# Programmation réseaux

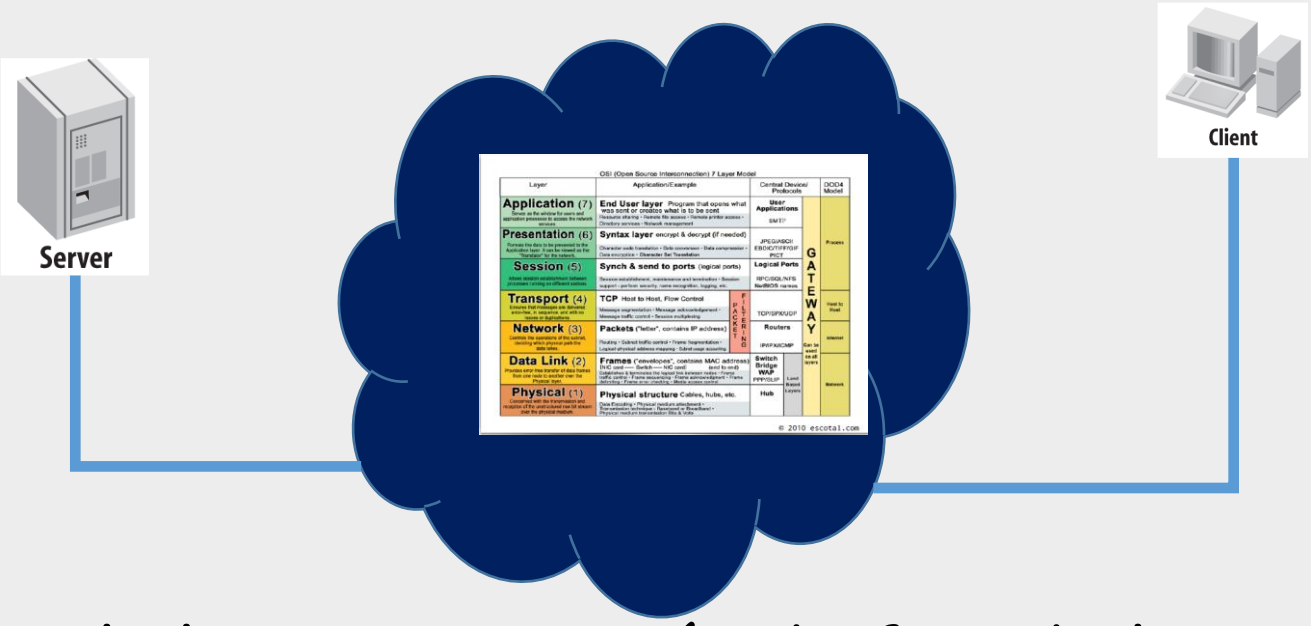
Master 1 CSIA

# Introduction

- Dans ce cours on s'intéresse à l'écriture d'applications qui interagissent (communiquent) à travers un réseau.
- Nous aurons besoin de quelques notions de base sur les réseaux informatiques.
- Mais, ce n'est pas les détails de fonctionnement des réseaux qui nous intéressent.
- Comprendre les mécanismes réseau
  - Communication
  - Codage de l'information
- Développer une application réseau
- Apprendre à programmer en Java et en Python

Une application voit le réseau comme un service, un moyen de communication, ou un espace d'une certaine nature.

- L'application ne s'intéresse pas aux détails de l'architecture du réseaux ou aux divers protocoles qui assurent son fonctionnement.




- Ces détails lui sont masqués du fait de la structure en couches de l'architecture des réseaux.

# Modèle de référence OSI/ISO

| | | | | | | |
|---|---|---|---|---|---|---|
| Data | **Application** (7) | **End User layer** Program that opens what was sent or creates what is to be sent | | **User Applications** SMTP | GATEWAY | Process |
| Data | **Presentation** (6) | **Syntax layer** encrypt & decrypt (if needed) | | JPEG/ASCII EBDIC/TIFF/GIF PICT | | |
| Data | **Session** (5) | **Synch & send to ports** (logical ports) | | **Logical Ports** RPC/SQL/NFS NetBIOS names | | |
| Segments | **Transport** (4) | **TCP** Host to Host, Flow Control | PACKET FILTERING | TCP/SPX/UDP | | Host to Host |
| Packets | **Network** (3) | **Packets** ("letter", contains IP address) | | **Routers** IP/IPX/ICMP | | Internet |
| Frames | **Data Link** (2) | **Frames** ("envelopes", contains MAC address) [NIC card — Switch — NIC card] (end to end) | | **Switch Bridge WAP** PPP/SLIP | Land Based Layers | Network |
| Bits | **Physical** (1) | **Physical structure** Cables, hubs, etc. | | **Hub** | | |

OSI (Open Source Interconnection) 7 Layer Model

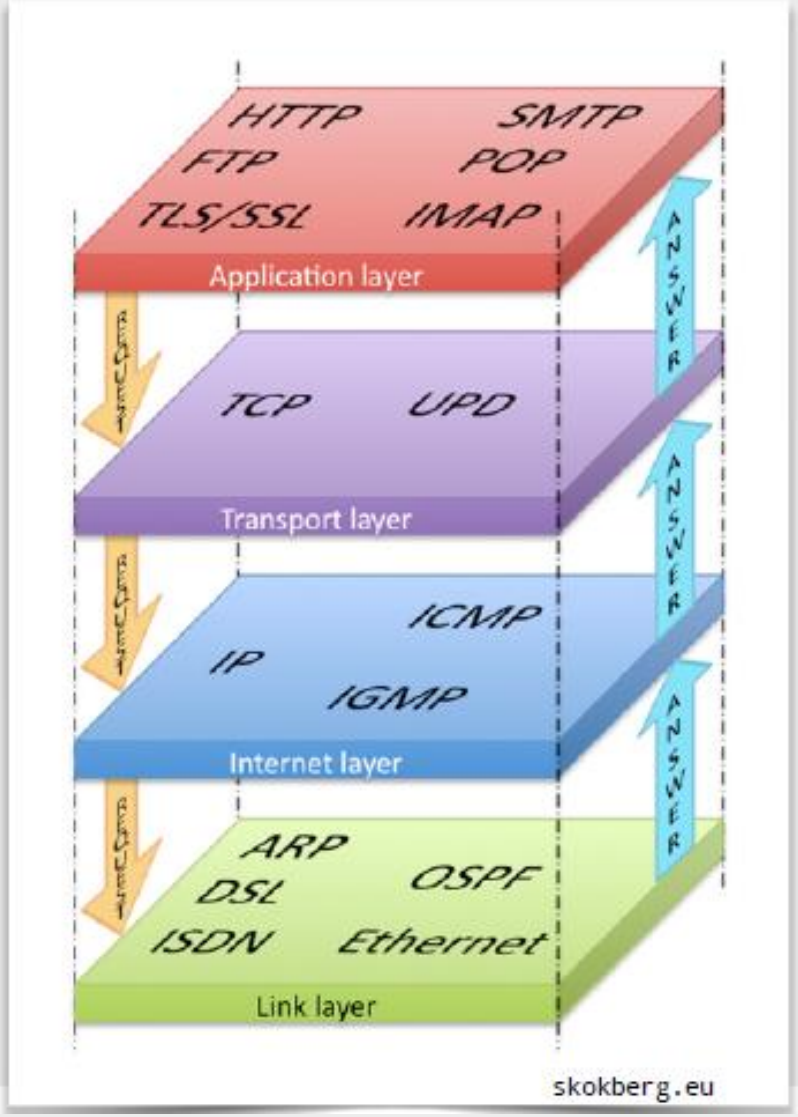
Layer | Application/Example | Central Device/ Protocols | DOD4 Model

Can be used on all layers

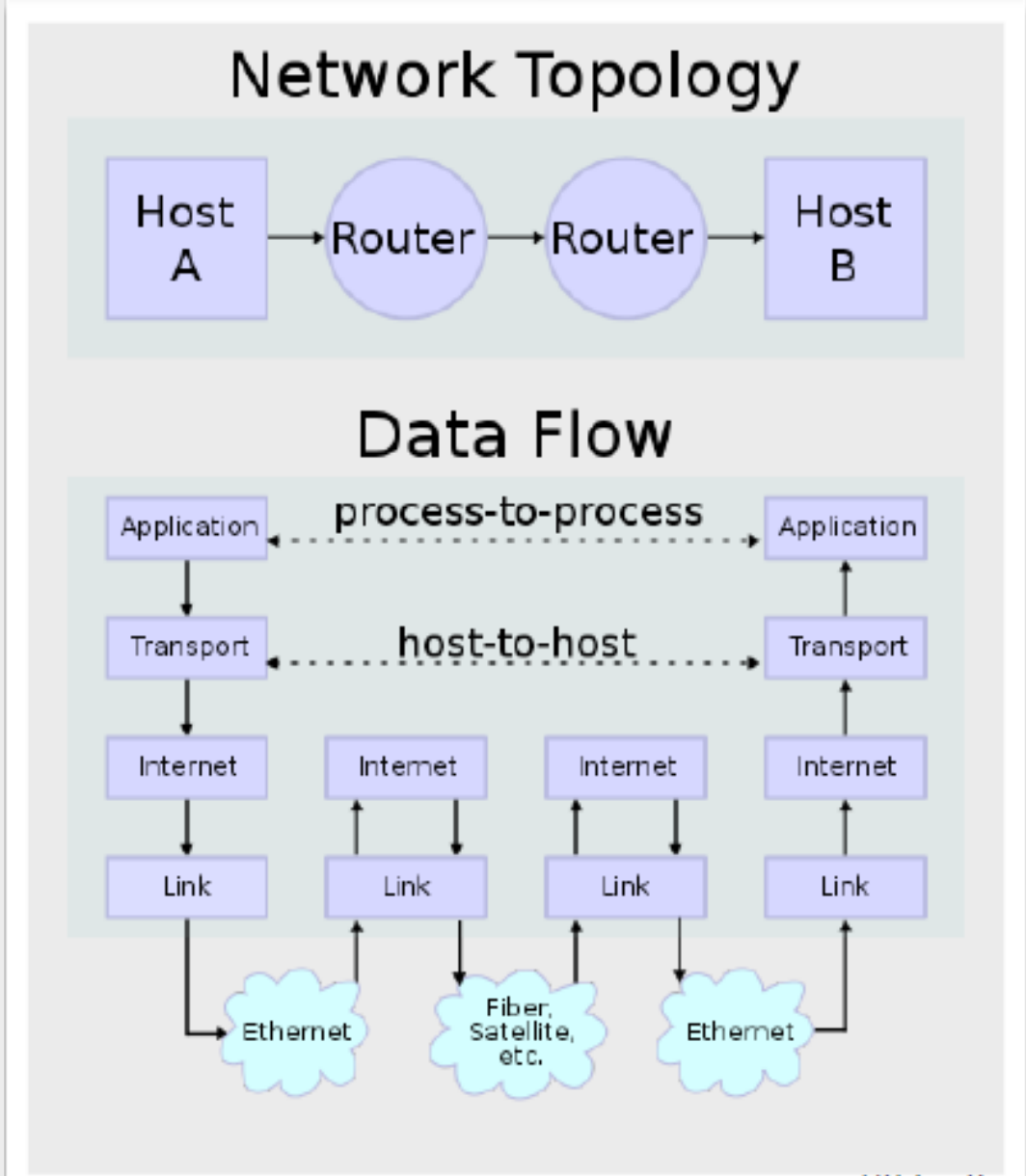
© 2010 escotal.com

| Couche | Protocoles |
| --- | --- |
| Application | FTP, HTTP, SMTP, POP, SSH, TELNET, IMAP… |
| Présentation | SSL, WEP, WPA, Kerberos |
| Session | Ports |
| Transport | TCP, UDP, SPX |
| Réseau | IPv4, IPv6, ARP, IPX |
| Liaison | 802.11, WiFi, ATM, Ethernet, ISDN |
| Physique | Fibre, Câble, Radio… |

- Le modèle Internet est à 4 couches



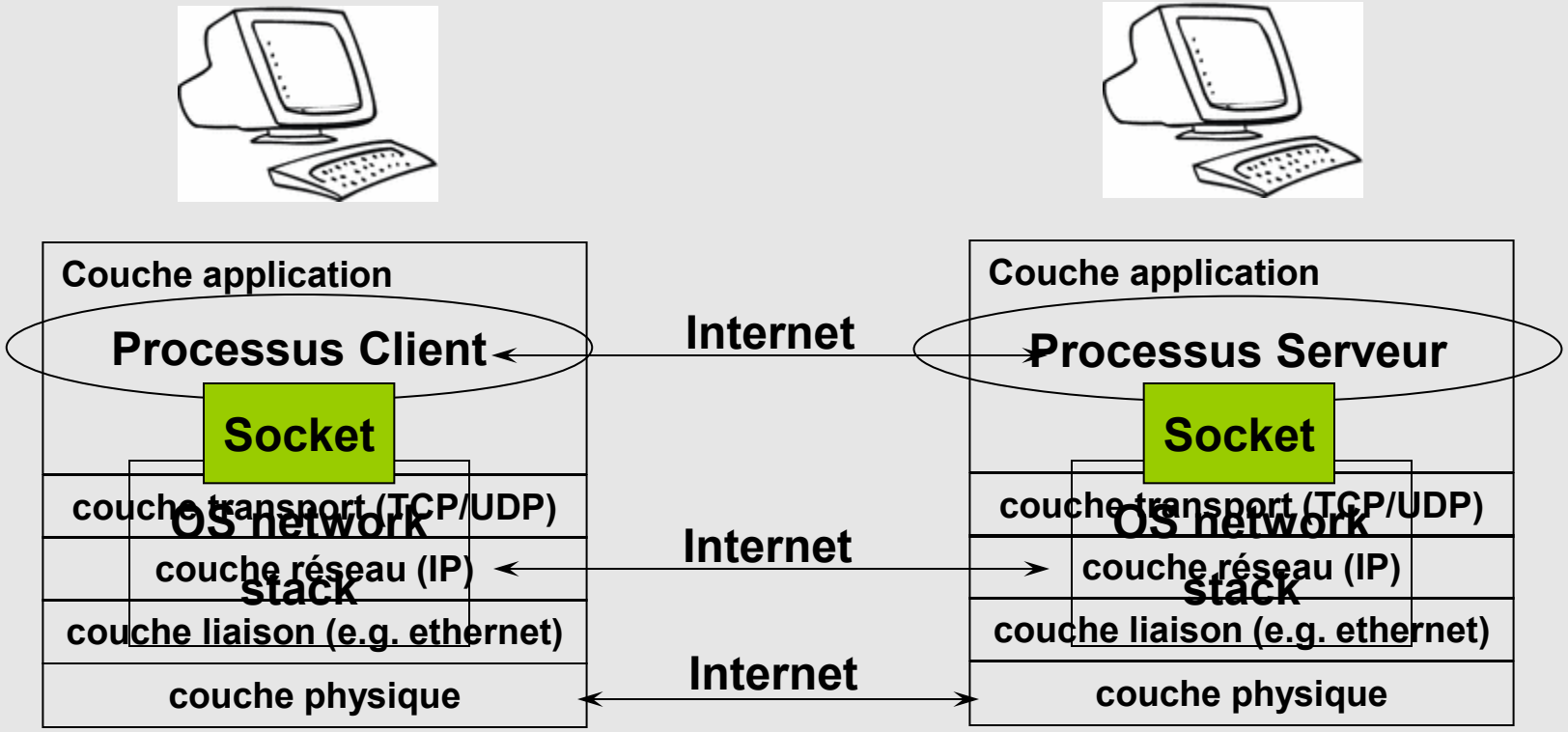
| Couche | Protocoles |
| --- | --- |
| Application | FTP, HTTP, IMAP, POP… |
| Transport | TCP, UDP… |
| Internet | IPv4, IPv6, IPsec, ICMP… |
| Liaison | ARP, PPP, DSL, Ethernet… |


Network Topology
Host A
Router
Router
Host B
Data Flow
Application
process-to-process
Application
Transport
host-to-host
Transport
Internet
Internet
Internet
Internet
Link
Link
Link
Link
Ethernet
Fiber, Satellite, etc.
Ethernet
Wikipedia

## Communications Client/Serveur

- Client demande (*requête*) / serveur fournit (*résponse*)
- Typiquement: un seul serveur / multiple clients
- Le serveur n'a pas besoin de connaître quoi que ce soit sur le client  (même pas qu'il existe)
- Le client doit toujours connaître quelque chose sur le serveur (au moins où il est localisé)

# Communication inter-processus (IPC)




L' interface que l'OS fournit à son sous-système réseau

## Communication inter-processus (IPC)

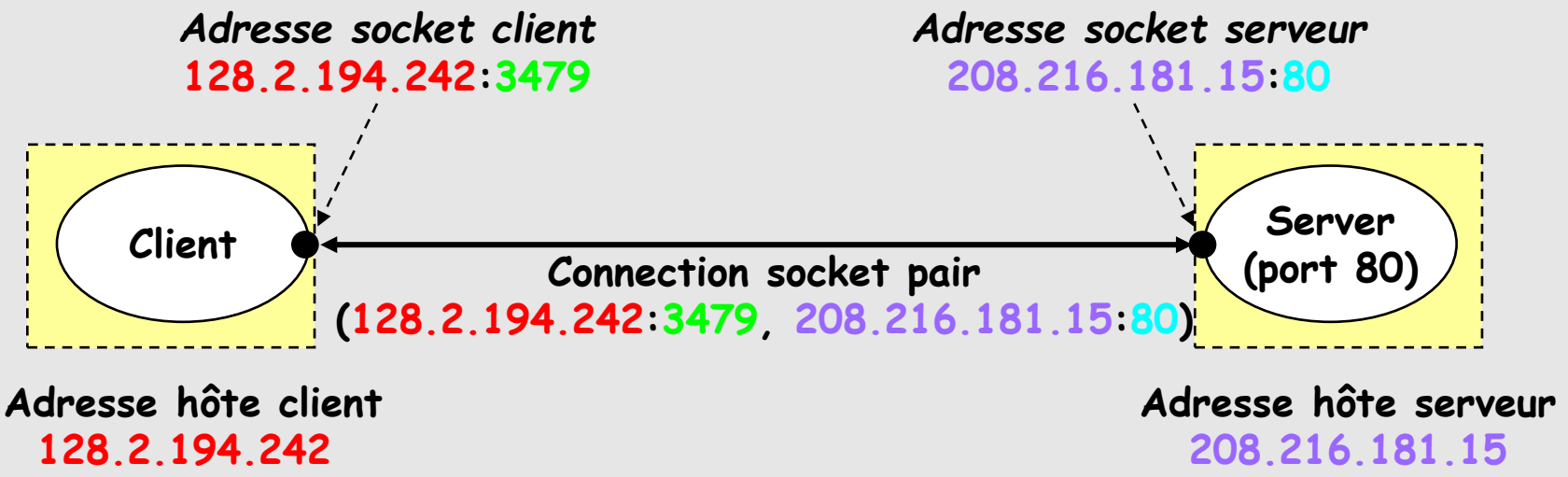
- La couche transport fournit un ou plusieurs services de communication entre applications (ex TCP ou UDP)
- Pour communiquer ces applications doivent se connaître
- Une application fournit un service particulier sur une machine donnée
- Le service est identifié par un port
- La machine est identifiée par une adresse

# Communication inter-processus (IPC)

- Un couple (adresse, port) est un point de communication
- Une communication s'effectue entre au moins deux points de communication(l'émetteur et le -ou les- récepteurs).

# Connections Internet (TCP/IP)

- Adresse de la machine sur le réseau
  - Par l'adresse IP
- Adresse du processus (ou service sur la machine)
  - Par le numéro de "port" – un nombre sur 16bits
- La pair *adresse IP + port* – constitue une "*adresse-socket*"



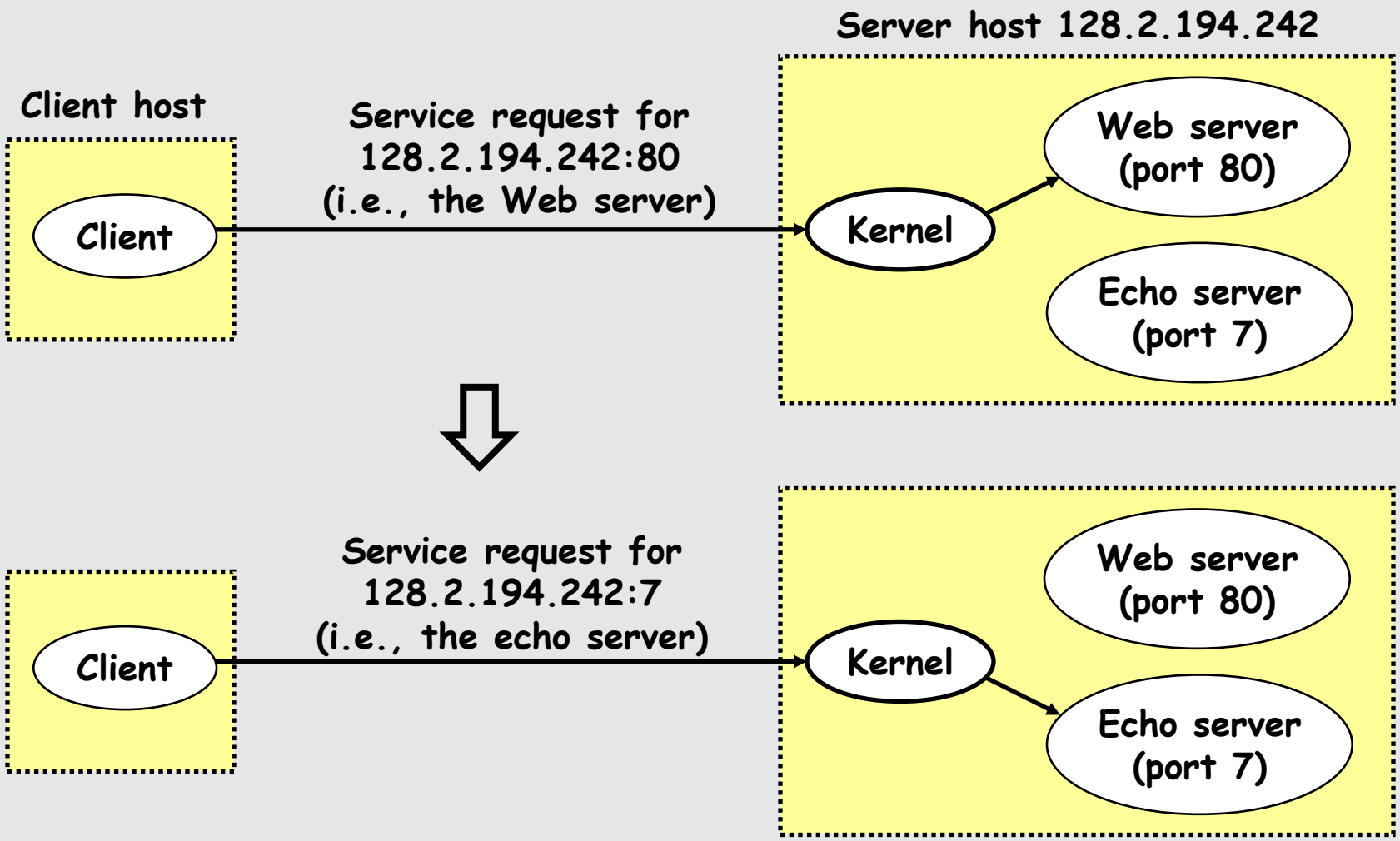

*Note: 3479 est un port éphémère alloué par le noyau*

*Note: 80 est un port bien connu associé avec des serveurs Web*

# Clients

- Examples de programmes clients
  - Web browsers, `ftp`, `telnet`, `ssh`
- Comment un client trouve-t-il le serveur?
  - L'adresse IP dans l'adresse socket du serveur identifie le hôte
  - Le (bien-connu) port dans l'adresse socket du serveur identifie le service, et ainsi identifie implicitement le processus serveur qui rend ce service.
  - Examples ports bien-connus (entre 1 et 1023)
    - Port 7: Echo server
    - Port 23: Telnet server
    - Port 25: Mail server
    - Port 80: Web server

# Utilisation des ports pour identifier les services



# Serveurs

- Les Serveurs sont des processus (deamons).
  - Crées au moment du boot (typiquement)
  - S'exécutent continuellement jusqu'à ce que la machine soit éteinte.
- Chaque serveur se met en attente de l'arrivée de requêtes sur un port bien-connu associé à un service particulier
  - Port 7: echo server
  - Port 23: telnet server
  - Port 25: mail server
  - Port 80: HTTP server
- Les autres applications doivent choisir un port entre 1024 and 65535